F FurniFlex

Project Documentation

A Full-Stack Furniture Rental Web Application — Architecture, Design, Database Schema and End-to-End Application Flow

PROJECT
FurniFlex — Furniture Rental Platform

REPOSITORY
github.com/RehanaPinjari22/furniflex

STACK
React (Vite) + Material UI · FastAPI + PostgreSQL

DOCUMENT DATE
August 2026

Built for a B.N.N College Project By Rehana Mansuri

Table of Contents

1. Introduction & Project Story
2. Scope of the Project
3. Technology Stack
4. System Architecture
5. Web Application Design (UI/UX)
6. Database Design & Schema
7. REST API Reference
8. End-to-End Application Flow
9. Security Measures
10. Deployment & Hosting
11. Known Gaps & Future Scope
12. Conclusion

1 Introduction & Project Story

1.1 Background

Buying furniture outright is expensive, inflexible, and inconvenient — especially for students, young professionals, and families who relocate often, live in rented apartments, or simply want to try a look before committing to it. Furniture-rental platforms such as **Furlenco** and **Rentomojo** solved this problem in the Indian market by letting customers rent quality furniture and appliances on flexible monthly plans instead of buying. **FurniFlex** is a full-stack clone-inspired implementation of that idea, built as a final-year college project to demonstrate end-to-end web application development — from database design to a production-style deployment pipeline.

1.2 Problem Statement

Design and build a web application that lets a customer browse furniture by category, choose a rental duration (Monthly, Quarterly, or Yearly), add items to a cart, check out, and track their orders — while giving an administrator full control over the product catalog, categories, orders and users through a dedicated admin panel.

1.3 Project Story

The project started from a single reference image (a mobile app mock-up in a warm amber/cream theme) and a feature list covering registration, browsing, cart, rental plans, checkout, order history and an admin panel. From there the application was built outward in layers: first a relational database schema and a REST API around it, then a themed React front-end consuming that API, then progressively richer features layered on top — a simulated payment step (Card / UPI / GPay QR), locally-hosted category-organized product images with an admin upload workflow, global search with filters, and a light/dark mode toggle. Each addition was verified end-to-end in a real browser against the live API before being considered complete, rather than assumed to work from the code alone.

1.4 Objectives

- Build a realistic, non-trivial full-stack application using an industry-standard stack (React + FastAPI + PostgreSQL).
- Design a normalized relational schema that correctly models a rental business (as opposed to a simple purchase e-commerce model).
- Implement secure authentication and role-based authorization (customer vs. admin).
- Provide a complete, usable admin back-office for day-to-day catalog and order management.
- Produce a responsive, themeable UI that works across devices and lighting preferences.
- Containerize the backend for reproducible local setup and discuss real deployment options.

2 Scope of the Project

2.1 In Scope

Customer-facing

- Registration & login (JWT-based)
- Home page with hero banner & category grid
- Browse furniture by category
- Product details with image gallery
- Global search + category/price filters + sort
- Rental plan selection (Monthly / Quarterly / Yearly)
- Cart management (add/update/remove)
- Checkout with simulated payment (Card / UPI / GPay QR)
- Order history
- Profile management
- Light / dark mode

Admin-facing

- Dashboard with key metrics
- Product CRUD with image upload
- Category CRUD with image upload
- Order listing across all customers
- Order status management
- User listing

Non-functional

- Responsive layout (mobile / tablet / desktop)
- Dockerized backend + database
- INR (₹) pricing throughout

2.2 Out of Scope

The following were deliberately excluded to keep the project focused on core full-stack fundamentals rather than third-party integrations:

- **Real payment processing** — the Card/UPI/GPay QR step is a UI simulation only; no payment gateway (Razorpay, Stripe, etc.) is integrated, and no real card data is ever stored.
- **Wishlist / favorites** — present in the original theme reference image but not in the written feature spec.
- **Email/SMS notifications** (order confirmations, OTP verification, password reset).
- **Product reviews & ratings.**
- **Multi-vendor support** — the catalog is single-tenant (one storefront, one admin team).

2.3 Target Users

Persona	Needs
Customer	Browse and rent furniture without a large upfront purchase; flexible duration; track orders.
Admin / Store Operator	Manage what's listed, at what price, in which category; fulfil and track orders; oversee registered users.

3 Technology Stack

3.1 Frontend

Layer	Technology	Purpose
Build tool	`Vite`	Fast dev server + production bundling
UI library	`React 19`	Component-based UI
Component kit	`Material UI (MUI) v7`	Themed components, responsive grid, dialogs, forms
Routing	`react-router-dom v7`	Client-side routing, protected/admin routes
HTTP client	`axios`	REST API calls, JWT header injection
Fonts	`@fontsource (Poppins, Inter)`	Self-hosted webfonts (no external CDN dependency)

3.2 Backend

Layer	Technology	Purpose
Language / runtime	`Python 3.12`	Backend application language
Web framework	`FastAPI`	Async REST API, automatic OpenAPI/Swagger docs
ORM	`SQLAlchemy 2.0`	Declarative models, typed `Mapped[]` columns
Validation	`Pydantic v2`	Request/response schemas, settings management
Auth	`python-jose + bcrypt`	JWT issuing/verification, password hashing
Server	`Uvicorn`	ASGI application server
Static files	`Starlette StaticFiles`	Serves category-organized product/category images

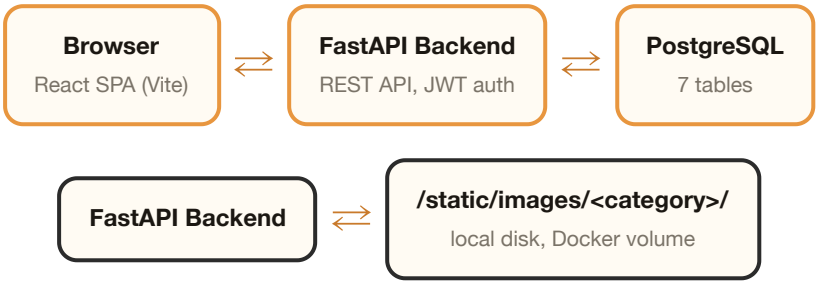
3.3 Database & Infrastructure

Component	Technology
Database	`PostgreSQL 16 (Alpine image)`
Containerization	`Docker + Docker Compose`
Version control	`Git / GitHub`

4 System Architecture

4.1 High-Level Architecture

FurniFlex follows a classic three-tier architecture: a React single-page application, a stateless FastAPI REST backend, and a PostgreSQL database — with a filesystem-backed static asset store for images.



The frontend never talks to the database directly — all data access goes through the versionless `/api/*` REST surface. Authentication is stateless: the backend issues a signed JWT on login/registration, the frontend stores it in `localStorage` and attaches it as a `Bearer` token on every subsequent request via an axios interceptor.

4.2 Backend Folder Structure

```
backend/
  app/
    models/       SQLAlchemy ORM models (7 tables)
    schemas/      Pydantic request/response models
    routers/      FastAPI route handlers, one file per resource
    core/           security.py (JWT + bcrypt), deps.py (auth dependencies)
    config.py         Settings (env vars) + static file paths
    database.py       SQLAlchemy engine/session factory
    seed.py             Demo data generator (categories, products, users)
    main.py               App entrypoint: CORS, routers, /static mount
  static/images/<slug>/  Category-organized product & category images
  requirements.txt
  Dockerfile
```

4.3 Frontend Folder Structure

```
frontend/
  src/
    components/
      layout/      Navbar, Footer, ProtectedRoute, AdminRoute
      product/       ProductCard, RentalPlanSelector
      admin/           AdminLayout, ImageUploader
      checkout/          PaymentMethodSelector
      common/              SearchBar, CategoryTile, LoadingSpinner
    pages/           Route-level pages (Home, CategoryPage, SearchResults,
```

```
                  ProductDetails, Cart, Checkout, OrderHistory, Profile,
                  Login, Register, and pages/admin/* for the back-office)
  context/        AuthContext, CartContext, ThemeModeContext
  services/        axios API client + one module per resource
  routes/           AppRoutes.jsx (route table)
  theme/              MUI theme (light + dark palettes)
```

5 Web Application Design (UI/UX)

5.1 Visual Identity

The visual language was derived from a mobile app reference image: a warm amber/mustard accent (`#E8963F`) over a soft cream base (`#FFF8EE`), rounded 20px cards, pill-shaped buttons, and Poppins/Inter typography. This was implemented as a reusable `createAppTheme(mode)` MUI theme factory supporting both a light palette and a dedicated dark palette (warm charcoal background `#17140F`, off-white text), toggled from the navbar and persisted in `localStorage`.

5.2 Page Catalog

Route	Page	Purpose
`/`	Home	Hero banner, category grid, featured products
`/categories`	Categories	Full list of all categories
`/categories/:slug`	Category Page	Products in a category with search, price filter, sort
`/search`	Search Results	Global search with category + price filters
`/products/:id`	Product Details	Gallery, rental plan selector, add to cart
`/cart`	Cart	Line items, quantity/plan editing, subtotal
`/checkout`	Checkout	Address, payment method, order summary
`/orders`	Order History	Past orders with status and payment info
`/profile`	Profile	Edit name/phone/address, logout
`/login, /register`	Auth	JWT login / account creation
`/admin`	Admin Dashboard	Key metrics: products, categories, users, orders, revenue
`/admin/products`	Admin Products	CRUD + image upload
`/admin/categories`	Admin Categories	CRUD + image upload
`/admin/orders`	Admin Orders	All orders, status updates
`/admin/users`	Admin Users	Registered user listing

5.3 Responsive Design

Layouts use MUI's breakpoint-driven `Grid` system (`xs / sm / md`) throughout — product grids collapse from 4 columns on desktop to 1 on mobile, the navbar swaps its inline links and search bar for a hamburger drawer + icon-only search under the `md` breakpoint, and forms stack vertically on small screens.

5.4 Access Control in the UI

Two route guard components gate the React Router tree: `ProtectedRoute` redirects unauthenticated users to `/login` (preserving their intended destination), and `AdminRoute` additionally checks `user.is_admin`, redirecting non-admins back to the home page. This is a UX convenience layer only — the backend independently enforces the same rules on every request, since client-side checks can always be bypassed.

6 Database Design & Schema

6.1 Entity Overview

PostgreSQL database with 7 tables. Legend: PK primary key, FK foreign key.



Relationships

- **users 1—* orders** — a user has many orders (cascade delete)
- **users 1—* cart_items** — a user has many cart items (cascade delete)
- **categories 1—* products** — a category has many products (cascade delete)
- **products 1—* cart_items / order_items**
- **rental_plans 1—* cart_items / order_items**
- **orders 1—* order_items** — an order has many line items (cascade delete)

6.2 Full Table Definitions

users

Column	Type	Constraints

id	Integer	Primary Key
name	String(120)	Not Null
email	String(255)	Unique, Indexed, Not Null
hashed_password	String(255)	Not Null (bcrypt hash, never plaintext)
phone	String(20)	Nullable
address	String(500)	Nullable
is_admin	Boolean	Default false
created_at	DateTime (tz-aware)	Default now()

categories

Column	Type	Constraints
id	Integer	Primary Key
name	String(100)	Unique, Not Null
slug	String(100)	Unique, Not Null — also the image folder name
description	String(500)	Nullable
image_url	String(1000)	Nullable — e.g. `/static/images/bedroom/img-1.jpg`

rental_plans

Column	Type	Constraints
id	Integer	Primary Key
name	String(50)	Unique — Monthly / Quarterly / Yearly
duration_months	Integer	Not Null — 1 / 3 / 12
discount_percent	Numeric(5,2)	Default 0 — 0% / 10% / 25%

products

Column	Type	Constraints
id	Integer	Primary Key
name	String(200)	Not Null
category_id	Integer	Foreign Key → categories.id
description	Text	Not Null
image_urls	JSON	List of image paths, default []
monthly_price	Numeric(10,2)	Not Null (INR)
quarterly_price	Numeric(10,2)	Not Null (INR, ~10% cheaper than 3× monthly)

yearly_price	Numeric(10,2)	Not Null (INR, ~25% cheaper than 12× monthly)
deposit	Numeric(10,2)	Default 0 — refundable security deposit
stock_qty	Integer	Default 10
is_featured	Boolean	Default false — drives Home page "Featured" section
created_at	DateTime (tz-aware)	Default now()

cart_items

Column	Type	Constraints
id	Integer	Primary Key
user_id	Integer	Foreign Key → users.id
product_id	Integer	Foreign Key → products.id
rental_plan_id	Integer	Foreign Key → rental_plans.id
quantity	Integer	Default 1

orders

Column	Type	Constraints
id	Integer	Primary Key
user_id	Integer	Foreign Key → users.id
status	String(30)	Default "PLACED" — PLACED / CONFIRMED / DELIVERED / RETURNED / CANCELLED
total_amount	Numeric(10,2)	Not Null (INR)
shipping_address	String(500)	Not Null
payment_method	String(20)	Default "CARD" — CARD / UPI / GPAY_QR
payment_reference	String(100)	Nullable — masked reference only, e.g. "Card ending 1234"
created_at	DateTime (tz-aware)	Default now()

order_items

Column	Type	Constraints
id	Integer	Primary Key
order_id	Integer	Foreign Key → orders.id
product_id	Integer	Foreign Key → products.id
rental_plan_id	Integer	Foreign Key → rental_plans.id
quantity	Integer	Default 1

price_at_order	Numeric(10,2)	Not Null — price snapshot at checkout time

> **Design note: why `price_at_order` exists**
> Product prices can change after an order is placed (an admin might edit them). Copying the price into `order_items.price_at_order` at checkout time means historical orders always reflect what the customer actually paid, independent of later catalog edits — a standard pattern in e-commerce/rental schema design.

6.3 Seed Data

The database is seeded with realistic demo data on first run:

- **9 categories**: Bedroom, BHK Combos, Living Room, Steal Deals, Storage, Kids, Study, Dining, Luxury
- **63 products** spread across those categories, each with 2 images, INR pricing, and a refundable deposit
- **3 rental plans**: Monthly (0% discount), Quarterly (10% discount), Yearly (25% discount)
- **2 demo accounts**: one admin, one regular customer

7 REST API Reference

All endpoints are prefixed `/api`. Interactive Swagger documentation is auto-generated by FastAPI at `/docs`.

Auth — /api/auth

Method	Path	Auth	Description
POST	`/register`	—	Create account, returns JWT + user
POST	`/login`	—	Email/password login, returns JWT + user
GET	`/me`	User	Current authenticated user

Users — /api/users

Method	Path	Auth	Description
PUT	`/me`	User	Update own profile (name, phone, address)

Categories — /api/categories

Method	Path	Auth	Description
GET	`/`	—	List all categories
GET	`/{id}`	—	Get one category
POST	`/`	Admin	Create category
PUT	`/{id}`	Admin	Update category
DELETE	`/{id}`	Admin	Delete category (cascades to its products)

Products — /api/products

Method	Path	Auth	Description
GET	`/`	—	List/search/filter products — query params: `category_id, category_slug, search, min_price, max_price, featured, sort`
GET	`/{id}`	—	Get one product with category
POST	`/`	Admin	Create product
PUT	`/{id}`	Admin	Update product
DELETE	`/{id}`	Admin	Delete product

Rental Plans — /api/rental-plans

Method	Path	Auth	Description
GET	`/`	—	List Monthly/Quarterly/Yearly plans

Cart — /api/cart

Method	Path	Auth	Description
GET	`/`	User	Current user's cart items
POST	`/`	User	Add item (merges quantity if same product+plan already in cart)
PUT	`/{item_id}`	User	Update quantity or rental plan
DELETE	`/{item_id}`	User	Remove one item
DELETE	`/`	User	Clear entire cart

Orders — /api/orders

Method	Path	Auth	Description
POST	`/checkout`	User	Converts cart → order, snapshots prices, clears cart
GET	`/`	User	Current user's order history
GET	`/{id}`	User/Admin	Order detail (owner or any admin)

Admin — /api/admin (all endpoints admin-only)

Method	Path	Description
GET	`/dashboard`	Totals: products, categories, users, orders, revenue
GET	`/users`	List all registered users
GET	`/orders`	List all orders across all customers
PATCH	`/orders/{id}/status`	Update order status
POST	`/upload-image`	Multipart image upload → saved to `static/images/<category_slug>/`

8 End-to-End Application Flow

8.1 Customer Journey

1. **Register / Login** — customer submits credentials; backend hashes/verifies password with bcrypt and issues a JWT; frontend stores it and attaches it as a Bearer token on every future request.
2. **Browse** — customer explores categories from the Home page or searches globally; `GET /api/products` supports category, price and text filters plus sort.
3. **View Product** — product detail page loads the item and the 3 rental plans; customer picks a plan and quantity.
4. **Add to Cart** — `POST /api/cart` creates or merges a cart line for that product + plan combination.
5. **Checkout** — customer enters/confirms a shipping address and picks a payment method (Card / UPI / GPay QR, simulated).
6. **Place Order** — `POST /api/orders/checkout` snapshots each line's price into `order_items.price_at_order`, creates the order, and empties the cart in a single transaction.
7. **Track Order** — customer reviews past orders and their status on the Order History page; the admin updates status as it's fulfilled.

8.2 Admin Journey

1. **Login** — same JWT flow, but the account has `is_admin = true`, unlocking the Admin Panel link and all `/api/admin/*` and admin-only CRUD endpoints.
2. **Dashboard** — at-a-glance totals for products, categories, users, orders and revenue.
3. **Manage Catalog** — create/edit/delete categories and products; upload images directly from the browser, which land in the matching `static/images/<category-slug>/` folder on the server.
4. **Fulfil Orders** — view every customer's orders and progress each one through PLACED → CONFIRMED → DELIVERED (or RETURNED / CANCELLED).
5. **Oversee Users** — view all registered customers and admins.

8.3 Image Upload Flow (Admin)

1. Admin selects a category for the product (or types a slug for a new category) and clicks the image tile.
2. Browser sends the file as `multipart/form-data` to `POST /api/admin/upload-image` with the target `category_slug`.

3. Backend validates: caller is admin, extension is one of jpg/jpeg/png/webp/gif, file ≤ 5MB, and the slug matches `^[a-z0-9-]+$` (blocks path traversal).
4. File is saved as `static/images/<slug>/<uuid>.<ext>`; the served URL is returned and attached to the product/category automatically.

9 Security Measures

- **Password hashing** — passwords are hashed with `bcrypt` before storage; plaintext passwords are never persisted or logged.
- **JWT authentication** — stateless bearer tokens signed with a server-side secret (`JWT_SECRET_KEY`), expiring after a configurable window (default 24h).
- **Role-based authorization** — a `get_current_admin` FastAPI dependency gates every admin-only route (product/category writes, all of `/api/admin/*`) independently of the frontend's UI guards.
- **Ownership checks** — a customer can only see/modify their own cart and orders; `GET /api/orders/{id}` allows the owning user or any admin, and no one else.
- **Upload hardening** — the image upload endpoint whitelists file extensions, caps size at 5MB, and validates the destination folder name against a strict regex to prevent directory traversal.
- **CORS** — restricted to the configured frontend origin(s) via `CORS_ORIGINS`, not left open (`*`).
- **No real payment data** — the checkout payment form is a simulation; only a masked reference (e.g. "Card ending 1234", or a UPI ID) is ever persisted, never a full card number or CVV.
- **Secrets via environment** — `DATABASE_URL`, `JWT_SECRET_KEY` etc. are read from environment variables / `.env` (gitignored), never hardcoded into source that reaches version control.

10 Deployment & Hosting

10.1 Local / Development Setup

The backend and database are containerized via Docker Compose for a reproducible one-command start:

```
docker compose up -d          # Postgres + FastAPI backend
cd frontend && npm install && npm run dev   # Vite dev server
```

The backend container seeds the database automatically on first boot. Product/category images live in a bind-mounted volume (`./backend/static → /app/static`) so admin-uploaded files persist across container restarts and are visible directly on the host filesystem.

10.2 Production Hosting Options Considered

Option	Fit	Notes
Vercel / Netlify	Frontend only	Excellent for the Vite/React build; not suitable for the backend as-is because serverless functions have an ephemeral filesystem, which would break local image uploads and doesn't suit a persistent SQLAlchemy connection pool.
Render / Railway	Backend + DB	Builds directly from the existing `Dockerfile`; supports a persistent disk (keeps `static/images` working) and managed Postgres.
AWS EC2	Full stack	A plain Linux VM can run `docker compose up -d` unmodified; trades PaaS convenience for manual ops (security groups, reverse proxy, TLS certificates, OS patching).

Recommended split for this project: frontend on Vercel/Netlify, backend + PostgreSQL on Render or Railway, connected via `VITE_API_BASE_URL` and `CORS_ORIGINS` environment variables.

10.3 Source Control

The project is version-controlled with Git and hosted at `github.com/RehanaPinjari22/furniflex`. Environment files, dependency directories (`node_modules`, Python virtualenvs), caches and local tool configuration are excluded via `.gitignore`.

11 Known Gaps & Future Scope

The following were identified during development as reasonable next steps, but are intentionally out of the current build (see §2.2 for what was deliberately excluded from day one):

Not yet implemented

- **Wishlist / Favorites**
  — present in the original visual reference (heart icon on product cards) but not built, since it wasn't in the written feature list.
- **Deeper admin user management**
  — admins can currently view all users but cannot edit roles, deactivate, or delete accounts from the UI.
- **Real payment gateway integration**
  — Razorpay/Stripe-style processing, to replace the current UI simulation.
- **Password reset / email verification**
  — no "forgot password" flow or email confirmation on signup.
- **Product reviews & ratings.**
- **Customer-initiated order cancellation**
  — currently only an admin can change order status; a customer cannot cancel their own pending order.
- **Pagination**
  on product/order listings — acceptable at the current demo scale (63 products) but would be needed at production scale.
- **Automated test suite**
  — the application was verified through manual and scripted end-to-end browser testing during development, but no unit/integration test suite is checked into the repository.

Recommended priority if extending this project

1) Automated tests, 2) real payment gateway, 3) wishlist, 4) password reset, 5) admin user management, 6) reviews & ratings, 7) pagination.

12 Conclusion

FurniFlex demonstrates a complete, working full-stack web application built around a realistic business domain — furniture rental rather than outright purchase — which required a schema and pricing model (rental plans, discount tiers, refundable deposits, price-at-order snapshots) beyond a typical CRUD e-commerce tutorial. The project covers the full stack expected of a final-year submission: a normalized relational database, a documented REST API with JWT authentication and role-based authorization, a themed and responsive React front-end with light/dark mode, a functioning admin back-office with real file uploads, and a containerized deployment story with a clear path to production hosting.

The codebase, database schema, and this document were all produced and verified together — every major feature (auth, cart/checkout, admin CRUD, image upload, search/filter, theming) was exercised end-to-end against the live API in a real browser before being considered complete, rather than assumed correct from source code alone.